FILKOM

Sorting

Advanced Programming

Ir. Aryo Pinandito, Ph.D.

Learning Outcomes

- Understand the concept of sorting algorithm
- Understand different types of sorting algorithm and how it works to sort numeric data
- Able to implement and apply three different types of sorting algorithm in the form of computer program and use it in a case-study.

Definition

- Sorting is one important algorithm for processing numeric or character data.
- Can be done in ascending or descending
- Sorting is a process to recompose data of a particular order or pattern, thus follows a particular rule in ordered manner.
- Algorithm to place a group of data elements into a particular order according to one or several keys of each element (*Microsoft Bookshelf*).

Sorting Types

- Ascending
  - Begins from the smallest key value and continue to ascend to the largest.
- Descending
  - Begins from the largest key value down to the smallest.

Example

- Numeric data:

  5, 2, 6, and 4

- Sorted in ascending:

  2, 4, 5, 6

  Or in descending:

  6, 5, 4, 2

Benefits of Sorting

- It reduces the complexity of a problem
- Data becomes easier to search (as in dictionary, phone book), ease to update, delete, insert, or merge
- Easier to check for missing items/elements
- Compiles a computer program in forming tables of symbols (*variables*)
- Improve data searching process that have to be performed repeatedly.

FILKOM

Factors That Affect Sorting Process Efficiency

- The amount of data to sort
- Storage capacity
- Storage media

Sorting Method

Sorting Method

- Bubble Sort
- Insertion Sort
- Selection Sort
- Quick Sort and Merge Sort
  - Sorting with Divide-and-Conquer technique (not discussed in this topic)
- Heap Sort (also not discussed)

Bubble Sort

- The easiest sorting method to understand
- Called "bubble" because of its sorting method that move each element sequentially to its correct position
  - Like a bubble that moving upward from the bottom of a soda bottle.
- Bubble sort compares each data element by comparing one element with another element next to it.

Bubble Sort

- Ascending bubble sort:
  - If next element (on the right) is smaller than current element, switch the element
- Descending bubble sort:
  - If next element (on the right) is larger than current element, switch the element
- Put simply, if an element is not in the right position in the sequence, move it to the left or the right.

Bubble Sort

- When does the bubble sort algorithm stop?
  - After all of the elements has been compared and checked. (number of iteration: n × n-1)
  - And the expected order has been reached.

Sort Example

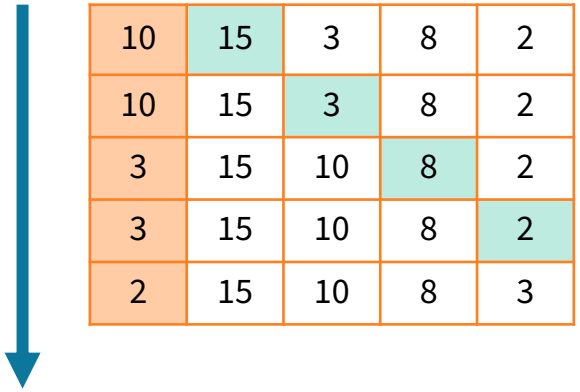
Given an array of integers with the following order:

10	15	3	8	2

Sort the given array
in ascending order!
using Bubble Sort

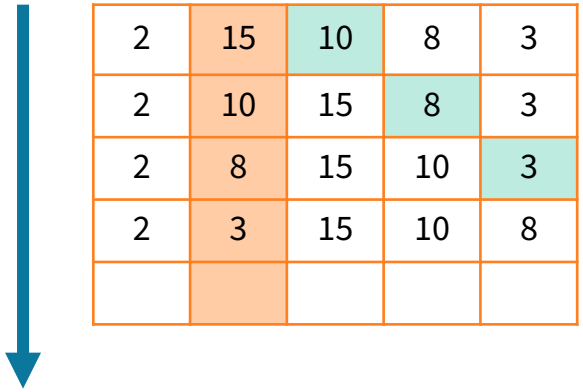
Bubble Sort Example

Iteration 1



10	15	3	8	2
10	15	3	8	2
3	15	10	8	2
3	15	10	8	2
2	15	10	8	3

Iteration 2



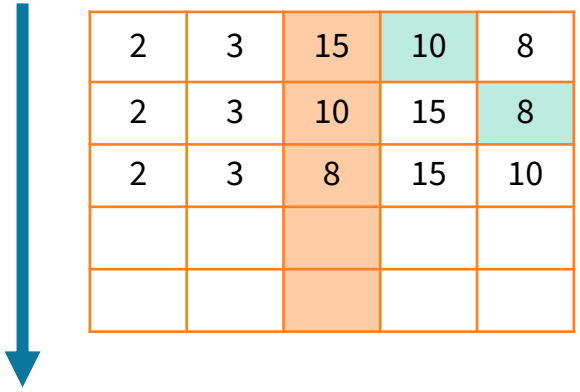
2	15	10	8	3
2	10	15	8	3
2	8	15	10	3
2	3	15	10	8



a	b

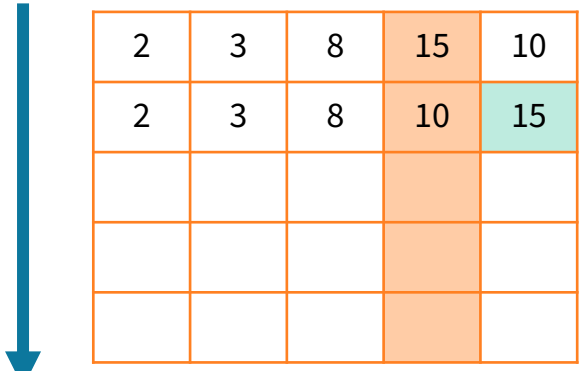
Comparison:
If b < a, then
switch a and b

Iteration 3



2	3	15	10	8
2	3	10	15	8
2	3	8	15	10

Iteration 4



2	3	8	15	10
2	3	8	10	15

FILKOM

Bubble Sort Implementation

```
static int[] data = {10, 15, 3, 8, 2};

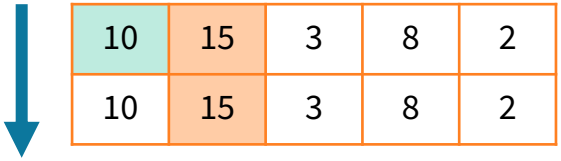
public static int[] bubbleSort(int[] data) {
  for (int i = 0; i < data.length; i++) {
    for(int j = i + 1; j < data.length; j++) {
      if (data[j] < data[i]) {
        int temp = data[j];
        data[j] = data[i];
        data[i] = temp;
      }
    }
  }
  return data;
}
```

Insertion Sort

- Similar in how people sort a card deck, one-by-one a card is picked and inserted in the correct position
- Sorting starts from 2nd element until the last element.
  - If smaller or larger element is found to its left, then found card is inserted in the correct position
- When a card is inserted, the other card will be shifted backward.

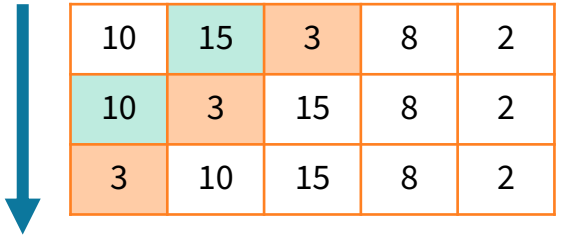
Insertion Sort Example

Iteration 1



10	15	3	8	2
10	15	3	8	2

Iteration 2

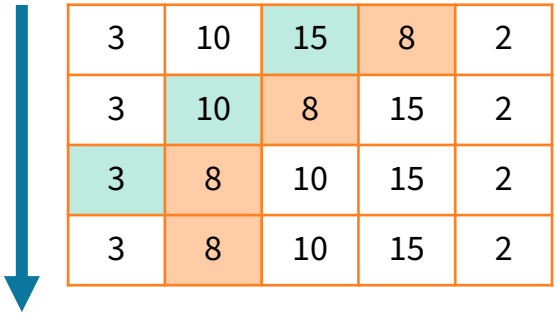


10	15	3	8	2
10	3	15	8	2
3	10	15	8	2

a	b

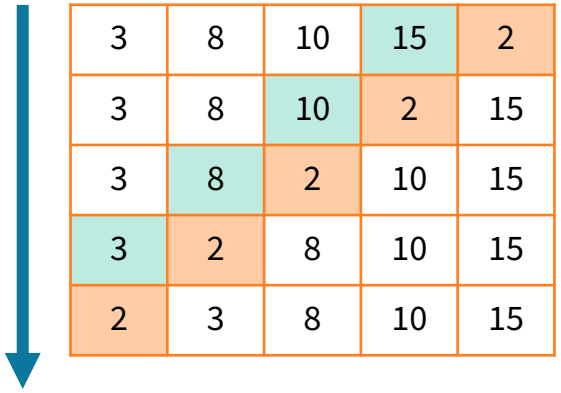
Comparison:
If b < a, then
switch a and b

Iteration 3



3	10	15	8	2
3	10	8	15	2
3	8	10	15	2
3	8	10	15	2

Iteration 4



3	8	10	15	2
3	8	10	2	15
3	8	2	10	15
3	2	8	10	15
2	3	8	10	15

FILKOM

Insertion Sort Implementation

```java
public static int[] insertionSort(int[] data) {

    for (int i = 1; i < data.length; i++) {
      int compare = data[i];
      int j = i-1;
      while (j >= 0 && data[j] > compare) {
        int temp = data[j+1];
        data[j+1] = data[j];
        data[j] = temp;
        j--;
      }
    }
    return data;
  }
```

Selection Sort

Selection sort is a combination between sorting and searching, hence the algorithm:

1. Starting from the first index (as the current index)
2. Find (the index) of an element from the unsorted part of n data whose value is minimum or maximum, and remember its position as the candidate to switch
3. If the value of current index is larger/smaller to the minimum/maximum value, switch the element at the current index with the element candidate to switch.
4. Next index becomes the current index; continue to step 2 until n becomes the current index.

Selection Sort Example

Iteration 1: Index 0

10	15	3	8	2

Min-value index	Compare Index	Compare Value	Switch Index?
0	1	15 > 10	No
0	2	**3 < 10**	**Yes**
2	3	8 > 3	No
2	4	**2 < 3**	**Yes**
4	-	-	

Switch value of Index 0 with index 4

2	15	3	8	10

Selection Sort Example

Iteration 2: Index 1

2	15	3	8	10

Min-value index	Compare Index	Compare Value	Switch Index?
1	2	**3 < 15**	**Yes**
2	3	8 > 3	No
2	4	10 > 3	No
2	-	-	-

Switch value of Index 1 with index 2

2	3	15	8	10

Selection Sort Example

Iteration 3: Index 2

2	3	15	8	10

Min-value index	Compare Index	Compare Value	Switch Index?
2	3	**8 < 15**	**Yes**
3	4	10 > 8	No
3	-	-	-

Switch value of Index 2 with index 3

2	3	8	15	10

Selection Sort Example

Iteration 4: Index 3

2	3	8	15	10

Min-value index	Compare Index	Compare Value	Switch Index?
3	4	**10 < 15**	**Yes**
4	-	-	-

Switch value of Index 3 with index 4

2	3	8	10	15

FILKOM

Selection Sort Implementation

```java
public static int[] selectionSort(int[] data) {
  int n = data.length;
  for (int i = 0; i < n-1; i++) {
    int minIdx = i;
    for (int j = i+1; j < n; j++)
      if (data[j] < data[minIdx])
          minIdx = j;

    int temp = data[minIdx];
    data[minIdx] = data[i];
    data[i] = temp;
  }
  return data;
}
```

Questions?